# SMART CONTACT MANAGEMENT

Assignment 2 of 14 — Services, Contracts & Schema

Design benchmark based on the supplied SCM source archive and CS543 assignment brief

**Deliverables:** capabilities, service design, service contracts, central operation specification, data schema and five-property validation.

**Architecture scope:** six business services behind an Express API Gateway; relational SQL plus MongoDB-oriented audit documents.

**Team members:** Aman Bali (20251651015) , Bhupesh kumar (20251651032)

# 1. Project overview and evidence boundary

Smart Contact Management lets a user register/sign in, manage a personal contact list, search and filter contacts, mark favorites, upload contact photos, export vCards, use contact parsing/email-drafting endpoints, receive email notifications, retain audit/contact-note documents, and call a SOAP/WSDL integration. The supplied README and source code identify an Express API Gateway and Java 21/Spring Boot services.

**Evidence caution:** the repository contains both service-specific Spring controllers and overlapping route logic in the gateway. The intended service boundaries are documented below, but the implementation is not fully isolated yet. The media store and notification dispatch log are in-memory; the audit repository also uses in-memory maps despite MongoDB configuration. Accordingly, this submission distinguishes existing contracts from target-state design recommendations.

# 2. Task 1 — Capability list

| Capability (verb phrase) | Description / evidence |
|---|---|
| Register and authenticate users | Local sign-up/login, JWT inspection/revocation, and Google/GitHub OAuth endpoints are documented and exposed by Auth. |
| Create and manage contacts | Create, read, replace, partially update, and delete contacts through /v1/contacts. |
| Search, filter, and paginate contacts | Search/favorite filtering and cursor-based pagination are implemented in ContactController. |
| Organize contacts into groups/segments | The repository documents contact groups and segment filters such as Favorites, VIP, Colleague, Lead, and Family; group persistence needs verification. |
| Upload and retrieve contact photos | Media endpoints validate photo content type and enforce a 2 MB maximum; current storage is process-local. |
| Export contacts as vCards | A vCard route is exposed for individual contacts. |
| Send welcome/recovery/transactional emails | Notification service accepts email dispatch requests and uses resilient fallback when SMTP is disabled or fails. |
| Record audit events and contact notes | Audit/document endpoints and MongoDB collection/index migration are present; current repository implementation is in-memory. |
| Parse contact text and draft emails | AI endpoints exist in Contact service and gateway; actual external AI dependency/configuration must be checked at runtime. |
| Integrate with SOAP/WSDL services | Integration service publishes WSDL/metadata, executes SOAP requests, and exposes resilience status. |

# 3. Task 2 — Service design and ownership

The API Gateway is a platform entry point, not counted as one of the six business services. Each business service owns its own data and exposes operations rather than allowing another service to query its tables directly. Cross-service calls use HTTP contracts; user identifiers carried between Auth and Contact are opaque references and must not be enforced through cross-service database foreign keys.

| Service / port | Owner | Exclusive responsibility and data ownership | Key operations |
|---|---|---|---|
| Auth Service<br/>Java 21, Spring Boot, Spring Security | [Aman bali] | Owns user identity, credentials, roles and OAuth identity metadata.<br/>Relational users table. Never expose password hashes. | POST /v1/auth/users; POST /v1/auth/tokens; GET /v1/auth/tokens/me; DELETE /v1/auth/tokens; OAuth routes. |
| Contact Service<br/>Java 21, Spring Boot, Spring Data JPA | [Aman bali] | Owns contact profiles, contact lifecycle, search, pagination and contact-group definitions.<br/>Relational contacts and contact_groups tables. userId is an opaque reference to Auth; no cross-service FK. | GET/POST /v1/contacts; GET/PUT/PATCH/DELETE /v1/contacts/{id}; GET /v1/contacts/{id}/vcard. |
| Media Service<br/>Java 21, Spring Boot | [Aman bali] | Owns uploaded contact-photo metadata and binary/object storage.<br/>Proposed durable media_assets metadata table; current Java implementation uses in-memory storage. | POST /v1/media/photos; GET /v1/media/photos/{id}; DELETE /v1/media/photos/{id}. |

| Service / port | Owner | Exclusive responsibility and data ownership | Key operations |
|---|---|---|---|
| Notification Service<br/>Java 21, Spring Boot, JavaMail | [Bhupesh Kumar] | Owns email dispatch requests and delivery status; SMTP failure should not block core workflows.<br/>Proposed durable notification_delivery table; current service keeps a process-local in-memory log. | POST /v1/notifications/emails; GET /v1/notifications/emails. |
| Audit Service<br/>Java 21, Spring Boot, MongoDB-oriented repository | [Bhupesh Kumar ] | Owns audit events, generic documents, contact notes and activity stream documents.<br/>MongoDB collections: audit-logs, contact-notes, activity-stream. Existing repository implementation currently uses in-memory maps despite Mongo configuration. | POST /v1/documents/{collection}; GET /v1/documents/{collection}; GET/DELETE /v1/documents/{collection}/{id}; contact notes routes via gateway. |
| Integration Service<br/>Java 21, Spring Boot | [Bhupesh Kumar] | Owns SOAP/WSDL integration behavior, external-call timeouts, retries and circuit-breaker state.<br/>Stateless; no business table required. External systems remain outside SCM ownership. | GET /v1/soap/wsdl; GET /v1/soap/metadata; POST /v1/soap/execute; GET /v1/resilience/status. |

See services.png for the service-call and ownership diagram. Gateway arrows represent routing to the public service contract; Auth → Notification is a best-effort notification interaction and should be asynchronous in the target architecture.

# 4. Task 3 — Service contracts

Contracts expose operation, inputs, outputs and caller-visible errors. Internal tables, identifiers used by storage, and SQL are deliberately excluded from these contract tables. Status codes below follow current controller behavior where visible; any target-state behavior is marked.

## Auth Service

| Operation | Input | Output | Caller-visible errors |
|---|---|---|---|
| registerUser | POST /v1/auth/users; name, email, password, optional phone/provider data | 201: public user profile and registration result | 400/422 invalid fields; 409 email conflict; 500 unexpected failure |
| issueToken | POST /v1/auth/tokens; email + password | 201: access token and token metadata | 401 invalid credentials; 404 account not registered; 422 malformed request |
| inspectToken | GET /v1/auth/tokens/me; Authorization: Bearer token | 200: token/user claims and status | 401 missing/invalid/expired token |
| revokeToken | DELETE /v1/auth/tokens; authenticated context if supported | 204 no content | 401 if authentication is required; note JWT revocation may be client-side/stateless |

## Contact Service

| Operation | Input | Output | Caller-visible errors |
|---|---|---|---|
| createContact | POST /v1/contacts; name, email, phoneNumber, optional address/description/favorite/links | 201: created contact and Location header | 400 malformed payload; 422 invalid email/phone; 401/403 unauthorized; 500 persistence error |
| listContacts | GET /v1/contacts; optional search, favorite, cursor, limit (1–100) | 200: items, nextCursor, hasMore, limit, totalCount | 400 invalid cursor/limit; 401/403 authorization failures |
| getContact | GET /v1/contacts/{id}; contact identifier | 200: contact representation in JSON/XML | 404 missing contact; 406 unsupported Accept type |
| updateContact | PUT/PATCH /v1/contacts/{id}; full or partial contact fields | 200: updated contact | 400 invalid payload; 404 missing contact; 422 invalid field values |
| deleteContact | DELETE /v1/contacts/{id} | 204 no content | 404 missing contact; 401/403 unauthorized |
| exportVCard | GET /v1/contacts/{id}/vcard | 200/redirect to vCard representation | 404 missing contact; 406 unsupported representation |

## Media Service

| Operation | Input | Output | Caller-visible errors |
|---|---|---|---|
| uploadPhoto | POST /v1/media/photos multipart file; <=2 MB, JPEG/PNG/WebP | 201: media id, file name, content type, size, URL, createdAt | 400 missing/empty file; 422 unsupported type or >2 MB; 500 storage error |
| getPhoto | GET /v1/media/photos/{id} | 200: photo metadata and cache headers | 404 unknown photo |
| deletePhoto | DELETE /v1/media/photos/{id} | 204 no content | 404 unknown photo; 403 if ownership policy rejects request |

## Notification Service

| Operation | Input | Output | Caller-visible errors |
|---|---|---|---|
| dispatchEmail | POST /v1/notifications/emails; to, type, optional subject/body | 201: id, recipient, type, status, dispatchedAt | 422 invalid recipient; SMTP errors are converted to fallback status rather than blocking caller |

| Operation | Input | Output | Caller-visible errors |
|---|---|---|---|
| listNotifications | GET /v1/notifications/emails | 200: list of dispatch records | 500 unexpected service error |

## Audit Service

| Operation | Input | Output | Caller-visible errors |
|---|---|---|---|
| createDocument | POST /v1/documents/{collection}; non-empty document | 201: stored document with id and creation metadata | 400 empty document; 404/422 unsupported collection or invalid document; 500 store error |
| listDocuments | GET /v1/documents/{collection} | 200: collection name, count, documents | 400 invalid collection; 500 store error |
| getDocument | GET /v1/documents/{collection}/{id} | 200: document | 404 document not found |
| deleteDocument | DELETE /v1/documents/{collection}/{id} | 204 no content | 404 document not found |

## Integration Service

| Operation | Input | Output | Caller-visible errors |
|---|---|---|---|
| getWsdl | GET /v1/soap/wsdl | 200: WSDL XML | 500 if WSDL cannot be produced |
| getWsdlMetadata | GET /v1/soap/metadata | 200: parsed service/operation metadata | 502 invalid/unavailable WSDL if applicable |
| executeSoap | POST /v1/soap/execute; operation/payload and optional SOAPAction | 200: normalized external SOAP result | 400 invalid request; 502 external SOAP failure; 504 deadline exceeded; 503 circuit open |
| resilienceStatus | GET /v1/resilience/status | 200: circuit-breaker state and counters | 500 unexpected service error |

# 5. Task 4 — Central operation specification: createContact

**Purpose.** Create one contact for the authenticated user and return a stable public representation. The endpoint is POST /v1/contacts. This is the central operation because contact creation is the core SCM domain action and it exercises validation, ownership, persistence, idempotency, and audit behavior.

## Inputs

| Field | Required? | Rules |
|---|---|---|
| Authorization | Yes for protected deployment | Bearer JWT; caller identity is derived from verified claims, not trusted from arbitrary body input. |
| name | Yes | Trimmed non-empty string; max 120 characters in the Java Contact entity. |
| email | Optional in current entity, commonly supplied | If supplied, validate email format; normalize whitespace and case for comparison/search where appropriate. |
| phoneNumber | Optional in DTO/entity but part of common create examples | If supplied, normalize to E.164 where country information is available; validate supported characters/length. |
| address / description | Optional | Address max 300 chars and description max 1000 in Contact entity. |
| favorite | Optional | Boolean; default false. |
| websiteLink / linkedInLink / pictureUrl | Optional | Validate URL format; pictureUrl should refer to Media-owned resource, not embed the file. |
| Idempotency-Key | Recommended for retry-safe POST | If gateway idempotency is enabled, identical key + same request returns the stored response; same key + different payload returns 409. |

## Preconditions and success response

1. The request body is valid JSON and passes schema validation. 2. Authentication/authorization is checked at the gateway/service boundary. 3. The owner identity is established from the verified principal. 4. Contact Service validates fields and writes only its own contact data. 5. On success return **201 Created**, a Location header pointing to /v1/contacts/{id}, and a JSON/XML contact representation containing id, userId, name, email, phoneNumber, address, description, favorite, links, pictureUrl, createdAt and updatedAt where applicable.

## Execution flow

**1.** Client sends POST /v1/contacts with JSON and optional Idempotency-Key.

**2.** API Gateway assigns/propagates correlation ID, applies rate limits, parses content type, validates the request schema, and performs authentication checks.

**3.** Gateway routes to Contact Service using the createContact contract; verified owner identity is passed as trusted context.

**4.** Contact Service validates business fields and checks ownership/authorization rules. It does not query the Auth database directly.

**5.** Contact Service persists the contact in its own datastore and commits the local transaction.

**6.** If idempotency is enabled, Gateway stores/replays the response in its own idempotency store. A best-effort audit event can be emitted after the core write; audit failure must not falsely report that the contact write failed after commit.

**7.** Return 201 with Location and contact representation; include correlation ID in logs/headers.

## Errors and failure behavior

| Condition | Response / behavior |
| --- | --- |
| Malformed JSON / missing required field | 400 Bad Request with stable problem-details body. |
| Schema or semantic validation fails | 422 Unprocessable Entity; include field-level detail without exposing internals. |
| Missing/invalid token or insufficient role | 401 Unauthorized / 403 Forbidden. |
| Idempotency key reused with different payload | 409 Conflict; do not create a second contact. |
| Contact does not exist during update (not create) | 404 Not Found. |
| Contact datastore unavailable before commit | 503/500 service error; no success response. Caller may retry with the same idempotency key. |
| Audit/notification dependency unavailable after contact commit | Do not roll back a committed contact across services. Record/log retryable event or outbox entry; return success for the contact if its primary write succeeded. |

## Internal details hidden from callers

Callers must not learn SQL/table names, database hosts, password/token signing secrets, storage paths, internal retry counters, stack traces, or whether the service uses MySQL versus another store. Expose stable field names, public IDs, HTTP status codes, correlation IDs and problem-details errors only. Cross-service work is not a distributed ACID transaction; use best-effort delivery or an outbox/event mechanism if durable audit/notification guarantees are required.

# 6. Task 5 — Database schema and ownership

The supplied repository has MySQL and PostgreSQL migration sketches and a MongoDB migration file. The README describes Aiven MySQL as the primary relational target and MongoDB for audit-oriented documents. To avoid mixing dialects, schema.sql uses MySQL 8.0 syntax. It is a target-state benchmark, not a claim that every listed table is already active.

| Owner | Data entities | Storage / constraints |
|---|---|---|
| Auth Service | Users | Relational; user_id PK, email unique, provider identity unique. Password hash only; never store plaintext passwords. |
| Contact Service | contacts, contact_groups, contact_group_members | Relational; contact_id PK; membership uses composite PK and same-service FKs. owner_user_id is an opaque Auth reference, no cross-service FK. |
| Media Service | media_assets metadata + binary object | Metadata table is proposed; binary file stored in media-owned object/file storage. 2 MB max and MIME allow-list. |
| Notification Service | notification_delivery_log | Proposed durable dispatch log; recipient, type, status and failure details owned only by Notification. |
| API Gateway platform | idempotency_keys | Gateway-owned metadata; key PK, request hash, stored response and expiry. Not shared with business services. |
| Audit Service | MongoDB: audit-logs, contact-notes, activity-stream | Document collections and indexes; no SQL CREATE TABLE. Current repository uses in-memory maps and requires durable Mongo integration to match intent. |
| Integration Service | None | Stateless SOAP/WSDL adapter; resilience state can be runtime state and need not be persisted for this assignment. |

## Schema relationship rules

• Auth owns users; no other service reads or writes the users table directly.

• Contact owns contacts and group membership. owner_user_id is a logical reference to Auth identity and is validated via contract/token claims, not a cross-service FK.

• contact_group_members has foreign keys only to Contact-owned tables.

• Media metadata is separate from Contact data. Contact stores only pictureUrl/reference.

• Notification stores delivery status separately; Auth asks Notification to dispatch an email rather than inserting into its data store.

• Audit collections contain copied identifiers/event context, not shared ownership of Contact or Auth records.

• Integration remains stateless; the remote SOAP system is not SCM-owned data.

The editable ER diagram is schema.drawio; schema.png is its exported image. schema.sql contains MySQL 8.0 CREATE TABLE statements for the relational target-state entities.

# 7. Task 6 — Five properties of a service

Assessment is based on source/configuration evidence where available. The target architecture is sound only when implementation boundaries match it; several current gaps are called out rather than being marked as passes.

| Service | Reachable | Self-contained | Has contract | Independent | Loosely coupled |
|---|---|---|---|---|---|
| Auth | PASS — port/controller routes exist. | PARTIAL — users schema/config exists; external OAuth config required. | PASS — REST controller and OpenAPI descriptions. | PARTIAL — gateway duplicates some auth routes; shared relational deployment possible. | PARTIAL — notification call should be async and secrets isolated. |
| Contact | PASS — REST routes and controller tests exist. | PARTIAL — entity/repository exist; user identity currently defaults in entity and must be tied to verified caller. | PASS — CRUD, paging and vCard routes. | FAIL/PARTIAL — overlapping gateway implementation and shared relational schema weaken ownership. | PARTIAL — media/audit dependencies should use contracts/events only. |
| Media | PASS — upload/get/delete controller routes exist. | FAIL — file bytes are held in process-local ConcurrentHashMap; restart loses data. | PASS — upload and metadata response are defined. | FAIL/PARTIAL — storage is not durable and deployment-specific. | PARTIAL — Contact should keep only a media URL/reference. |
| Notification | PASS — dispatch/list routes exist. | FAIL — dispatch log is in-memory; no durable queue/log. | PASS — request/response maps are exposed. | PARTIAL — SMTP config is optional and mail delivery is a best-effort fallback. | PASS/PARTIAL — service catches SMTP failures; async queue/outbox would improve it. |
| Audit | PASS — document controller routes exist. | FAIL — repository currently uses in-memory maps even though Mongo is configured. | PASS — generic document operations exist, but collection allow-list should be enforced. | FAIL/PARTIAL — audit routes/data are also implemented in gateway; duplicate ownership risk. | PARTIAL — audit should consume events without blocking primary writes. |
| Integration | PASS — WSDL/execute/resilience routes exist. | PASS/PARTIAL — mostly stateless; requires endpoint/config and external SOAP availability. | PASS — WSDL and JSON operation endpoints. | PARTIAL — external endpoint dependency remains; circuit breaker mitigates it. | PASS/PARTIAL — timeout, retries and circuit breaker reduce coupling; avoid unbounded retry. |

## Recommended fixes in priority order

**1.** Choose one authoritative route implementation: gateway should route to Java services instead of reimplementing domain endpoints in the same process.

**2.** Give each service exclusive persistence ownership. Split Auth and Contact schemas/credentials, and remove direct cross-service table access. Do not use cross-service FKs.

**3.** Replace Media ConcurrentHashMap storage with durable object storage and persist media metadata; validate file content as well as claimed MIME type.

**4.** Connect Audit repository to configured MongoDB; restrict collection names and enforce append-only semantics for audit events where required.

**5.** Persist Notification delivery state and preferably publish messages through a queue/outbox; maintain retry limits and dead-letter handling.

**6.** Ensure user identity comes from verified JWT claims, not hard-coded/default user IDs or caller-controlled fields.

**7.** Add contract tests between gateway and each service; keep OpenAPI definitions synchronized with real endpoints and errors.

**8.** Add health/readiness checks, timeouts, bounded retries, structured correlation IDs, and per-service monitoring.

# 8. Submission checklist

| Deliverable | Status in this package |
|---|---|
| design.pdf | This document; tasks 1, 3, 4 and 6 plus ownership summary. |
| services.drawio | Editable diagrams.net service diagram. |

| Deliverable | Status in this package |
| --- | --- |
| services.png | PNG export of service design. |
| schema.drawio | Editable diagrams.net schema/ownership diagram. |
| schema.png | PNG export of schema diagram. |
| schema.sql | MySQL 8.0 relational schema sketch. |